



GigConnect Database ER-Diagram & Application Workflow Guide

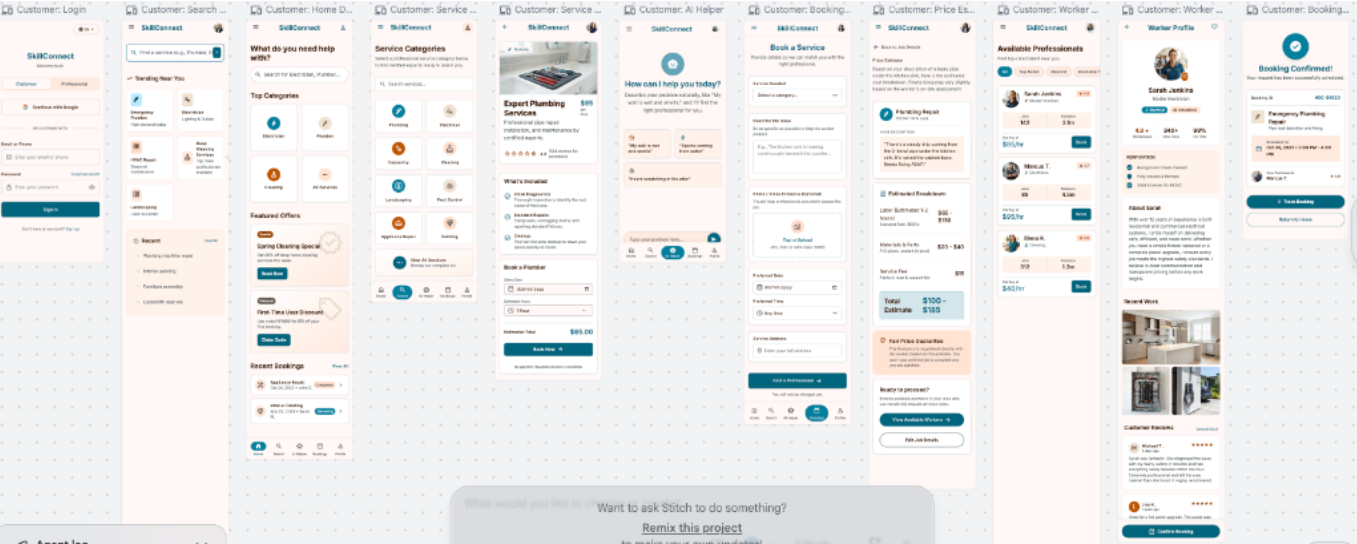
This document provides a detailed breakdown of the **Entity-Relationship (ER) Diagram, Schema Connections (Database Relations),** and the **End-to-End Application Workflow** matching the UI mockups.



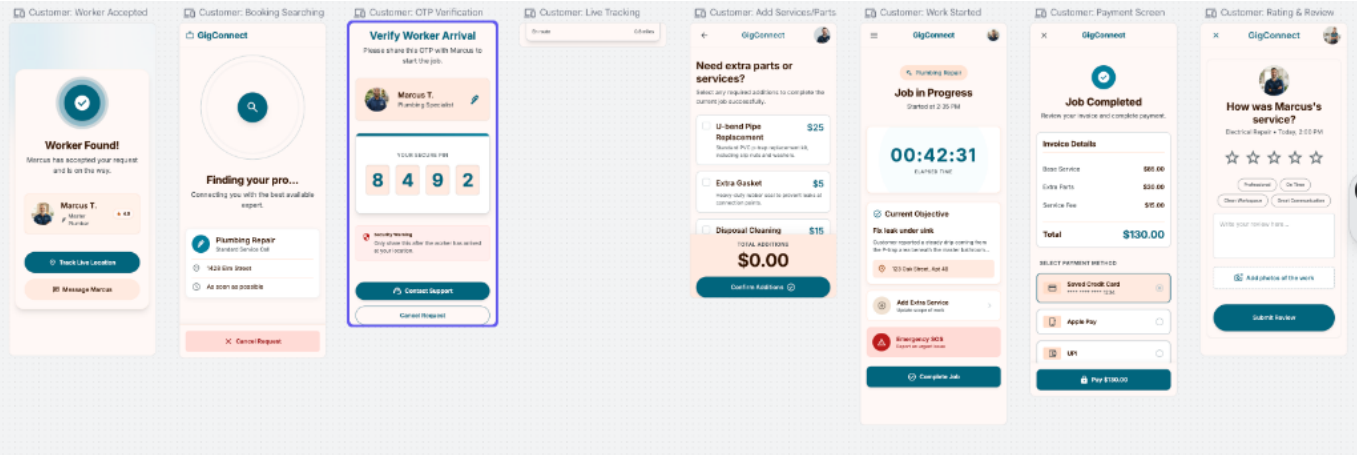
UI Reference Panels

Below are the UI panels from the application design screens for mapping the databases to the visual workflow:

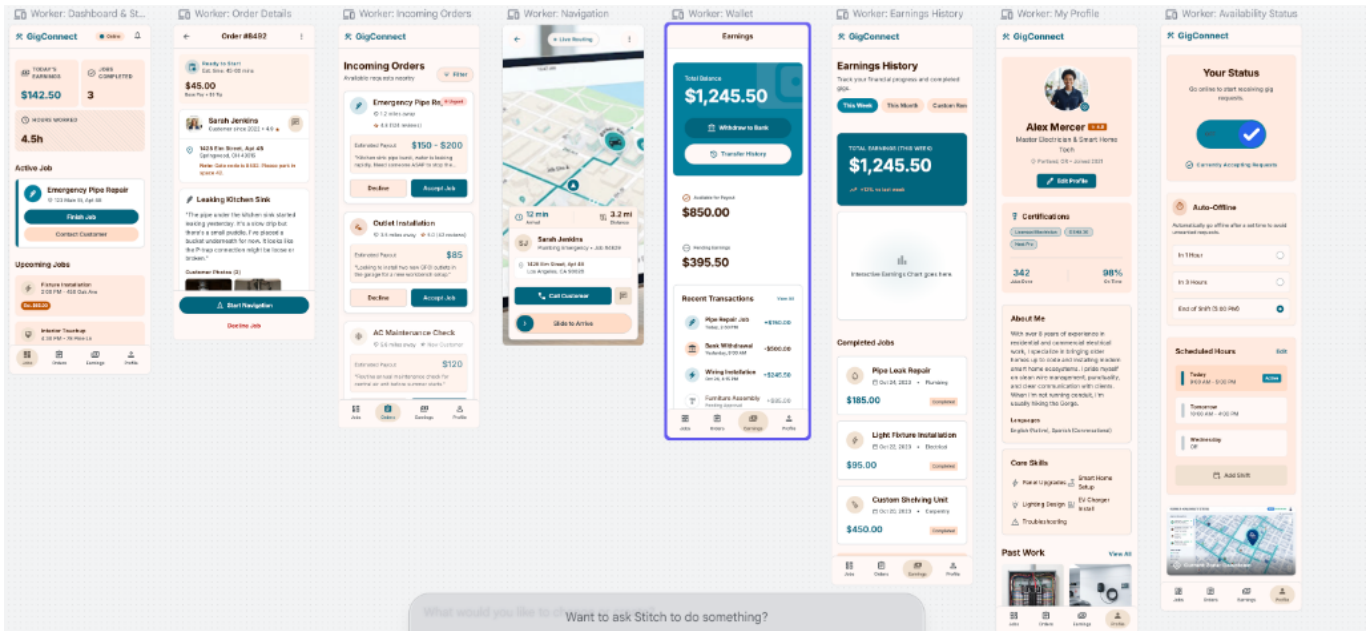
Panel 1: Customer Onboarding, Service Selection, and Worker Details



Panel 2: Live Tracking, Active Work Progress, and Payments



Panel 3: Worker Interface, Navigation, and Wallet Earnings



Entity-Relationship (ER) Diagram

The following diagram illustrates how all database schemas are connected inside MongoDB:

```
erDiagram
    USER {
        ObjectId _id PK
        string name
        string email
        string phone
        string role "customer | worker | admin"
        string authProvider "local | google"
        string avatar
        boolean isVerified
        Point location "GeoJSON coordinates [lng, lat]"
        array savedAddresses "nested addressSchema"
        object workerProfile "nested workerProfileSchema"
        string activeDeviceId
    }

    BOOKING {
        ObjectId _id PK
        string bookingId "unique e.g. #BK-84920"
        ObjectId customer FK "ref: User"
        ObjectId worker FK "ref: User (nullable)"
        ObjectId service FK "ref: Service"
        string status "SEARCHING | ACCEPTED | ARRIVED | IN_PROGRESS | COMPLETED | CANCELLED"
        string problemDescription
        array problemPhotos
        object serviceAddress
        Date scheduledTime
        string arrivalotp "4-digit Secure PIN"
        array addOns "nested addOnItemSchema [{title, price}]"
    }
```

```

    Date jobStartedAt
    Date jobCompletedAt
    boolean isReviewed
    object invoice "base, extra, platform, total"
  }

SERVICE {
  ObjectId _id PK
  string title "e.g. Expert Plumbing"
  string category "e.g. plumbing"
  string image
  number basePrice
  string estimatedTime
  array whatsIncluded
  boolean isActive
}

REVIEW {
  ObjectId _id PK
  ObjectId booking FK "ref: Booking"
  ObjectId customer FK "ref: User"
  ObjectId worker FK "ref: User"
  number rating "1 to 5"
  string feedback
  array badgesGiven "traits e.g. Professional, On Time"
  array photos
}

TRANSACTION {
  ObjectId _id PK
  ObjectId customer FK "ref: User"
  ObjectId worker FK "ref: User"
  ObjectId booking FK "ref: Booking"
  string orderId "Razorpay Order ID"
  string paymentId "Razorpay Payment ID"
  number amount
  string currency "default: INR"
  string status "pending | success | failed"
  string paymentMethod "default: Razorpay"
}

USER ||--o{ BOOKING : "creates (as customer)"
USER ||--o{ BOOKING : "performs (as worker)"
SERVICE ||--o{ BOOKING : "is booked for"
BOOKING ||--|| REVIEW : "is evaluated by"
USER ||--o{ REVIEW : "writes (as customer)"
USER ||--o{ REVIEW : "receives (as worker)"
BOOKING ||--|| TRANSACTION : "is paid via"
USER ||--o{ TRANSACTION : "pays (as customer)"
USER ||--o{ TRANSACTION : "earns (as worker)"

```

Detailed Schema Connections & Database Fields

1. Unified User Schema (**User** Collection)

MongoDB merges customer and worker details into a single collection to optimize authentication, logins, and session locks.

- **Role Differentiation:** Handled by the **role** enum (**'customer'**, **'worker'**, **'admin'**).
- **GeoJSON Indexing:** **location** maps worker/customer GPS coordinates using MongoDB **2dsphere** spatial index:

```
"location": { "type": "Point", "coordinates": [77.2090, 28.6139] }
```

- **Embedded Document (**workerProfile**):** For worker accounts, this contains skills, rating, certificates/badges, bio, and hourly rates (saving expensive collection joins on search):
 - **skills** (Array of Strings) - core capabilities (e.g. *Troubleshooting, Panel Upgrades*).
 - **certifications** (Array of Strings) - certifications (e.g. *Licensed Electrician, Nest Pro*).
 - **badges** (Array of Strings) - operational badges (e.g. *Background Checked, Top Rated*).
 - **rating** (Number) - average rating from reviews.
 - **totalJobs** (Number) - total completed bookings.

2. Booking Schema (**Booking** Collection)

Binds the customer, worker, and service catalog together to track the lifecycle of the job.

- **customer:** References **User** (**_id**) where **role** = **'customer'**.
- **worker:** References **User** (**_id**) where **role** = **'worker'**. Set to **null** while searching, populated upon acceptance.
- **service:** References the chosen task in the **Service** collection.
- **addOns:** Represents extra parts added during the job (e.g., U-bend pipe). Stores items directly as sub-documents in the booking.
- **invoice:** Nested object storing price breakdowns:
 - **baseServiceFee** (Hourly rate $\times$ duration or service base price)
 - **extraPartsTotal** (Sum of prices of all items in **addOns**)
 - **platformFee** (Flat platform commission)
 - **totalAmount** (Sum of **baseServiceFee** + **extraPartsTotal** + **platformFee**)

3. Review Schema (**Review** Collection)

Tracks feedback and traits given to workers after job completion.

- **Relationships:** Connects **booking** (1:1), **customer** (Many:1), and **worker** (Many:1).
- **badgesGiven:** Maps to the traits select UI list (e.g., *Professional, On Time, Good Communication*).

4. Transaction Schema (**Transaction** Collection)

Manages Razorpay checkout sessions.

- **orderId**: Tracks the order ID created by Razorpay.
- **paymentId**: Received from Razorpay checkout upon successful payment.
- **status**: Maps payments states ('pending', 'success', 'failed').

End-to-End Application Workflow

The diagram below details the operational workflow mapped directly to the UI screens in the panels:

```
sequenceDiagram
    autonumber
    actor Customer as  Customer
    actor Worker as  Worker
    participant Backend as  API Backend (Express)
    participant Redis as  Redis Cache
    participant Sockets as  Socket.io

    %% Phase 1: Authentication & Setup
    Note over Customer, Backend: Phase 1: Login & Authentication
    Customer->>Backend: POST /api/auth/login (email, password, deviceId)
    Backend->>Redis: Set user session lock (deviceId)
    Backend-->>Customer: Return accessToken & rotated refreshToken

    %% Phase 2: Booking Estimate & Creation
    Note over Customer, Backend: Phase 2: Estimation & Booking Creation
    Customer->>Backend: GET /api/home/home (fetch services list)
    Customer->>Backend: POST /api/bookings/estimate (serviceId, estimatedHours)
    Backend-->>Customer: Return Estimate Breakdown (Labor, Materials, Platform Fee)
    Customer->>Backend: POST /api/bookings (Create booking, status: SEARCHING)
    Backend-->>Customer: Return Booking Details & arrivalOtp (#BK-84920)

    %% Phase 3: Matching & Live Tracking
    Note over Worker, Sockets: Phase 3: Booking Matching & Live Navigation
    Worker->>Backend: POST /api/bookings/:bookingId/accept
    Backend->>Sockets: Emit 'booking_status_update' (status: ACCEPTED)
    Sockets-->>Customer: Received status update (Worker found!)

    Worker->>Sockets: Emit 'worker_location_update' (lat, lng)
    Sockets->>Backend: Save coordinates to User in MongoDB (async)
    Sockets-->>Customer: Emit 'live_tracking' (updates map position in real time)

    %% Phase 4: Verification & Arrival
    Note over Customer, Worker: Phase 4: Arrival Verification (OTP)
    Worker->>Customer: Arrives & asks for Secure PIN (arrivalOtp)
    Customer->>Worker: Shares PIN (e.g. 8492)
    Worker->>Backend: POST /api/bookings/:bookingId/verify-otp (otp)
    Backend->>Sockets: Emit 'booking_status_update' (status: ARRIVED)
    Sockets-->>Customer: Arrived state updated in UI
```

```
%% Phase 5: Execution & Addons
Note over Customer, Worker: Phase 5: Work in Progress & Extra Parts
Worker-->>Backend: POST /api/bookings/:bookingId/start-job
Backend-->>Sockets: Emit 'booking_status_update' (status: IN_PROGRESS)
Sockets-->>Customer: Timer starts in Customer app (Elapsed Time)

Worker-->>Backend: PATCH /api/bookings/:bookingId/add-parts
(extraItems)
Backend-->>Sockets: Emit 'booking_status_update' (updates invoice &
addons live)
Sockets-->>Customer: Customer approves add-ons in UI

%% Phase 6: Job Complete & Payment
Note over Worker, Backend: Phase 6: Completion & Checkout
Worker-->>Backend: POST /api/bookings/:bookingId/complete
Backend-->>Sockets: Emit 'booking_status_update' (status: COMPLETED)
Sockets-->>Customer: Redirect to payment checkout
Customer-->>Backend: POST /api/payments/verify (Razorpay verification)
Backend-->>Backend: Set booking invoice paymentStatus to PAID
Backend-->>Customer: Checkout success screen

%% Phase 7: Reviews & Profile update
Note over Customer, Backend: Phase 7: Review & Rating Sync
Customer-->>Backend: POST /api/reviews (rating, comment, traits)
Backend-->>Backend: Aggregate reviews & update workerProfile.rating
average
Backend-->>Customer: Feedback submitted!
```

How to Export this Document to PDF

To export this document as a clean, formatted PDF:

1. VS Code / IDE:

- Open this file in your IDE.
- Install the **Markdown PDF** extension.
- Right-click in the markdown file editor window and select **Markdown PDF: Export (pdf)**.

2. Web Browser (Chrome/Edge/Safari):

- Paste the contents of this file into any markdown editor viewer (e.g., [StackEdit](#)).
- Open the print menu in your web browser (**Ctrl+P** or **Cmd+P**).
- Choose **Save as PDF** as the destination and click **Save**.